

Aircraft Damage Detection and Captioning

Using Pretrained Deep Learning Models

A Technical Research Paper and Code-Level Analysis of the Aircraft-Dent-Detector Repository

Author: Manab Barman

Repository: <https://github.com/emManab/Aircraft-Dent-Detector>

Analysis date: 22 August 2026

This paper documents the implementation as it exists in the public repository and notebook, rather than presenting the system as a production-certified aviation inspection tool.

Abstract

Objective. This study analyzes a two-stage computer vision and vision-language pipeline implemented in the Aircraft-Dent-Detector repository. The first stage classifies aircraft surface images into dent or crack using a transfer-learning architecture built on ImageNet-pretrained VGG16. The second stage uses Salesforce BLIP to produce a general caption or a more inspection-oriented textual description. **Methods.** The notebook downloads and extracts an aircraft-damage dataset, organizes images into train/validation/test directories, rescales pixels to [0,1], resizes images to 224x224, trains a frozen VGG16 feature extractor with a custom dense classifier for five

epochs, and evaluates predictions. A custom Keras layer wraps BLIP so image paths and task labels can be passed through TensorFlow while the underlying generation runs with PyTorch Transformers. **Results.** The notebook reports 300 training images, 96 validation images, and 50 test images. Training accuracy rises from 53.67% to 88.00%, while validation accuracy peaks at 71.88% and ends at 70.83%. Reported test loss is 0.7326 and test accuracy is 68.75%; importantly, the evaluation call uses one full batch because it computes $\text{test_samples} // \text{batch_size}$ ($50 // 32 = 1$), so only 32 test images are scored by that call. **Conclusion.** The project is a useful educational prototype demonstrating transfer learning and multimodal reporting, but its current experimental design is not sufficient for safety-critical aircraft inspection. The analysis identifies the major strengths, implementation choices, reproducibility issues, and clear next steps such as fine-tuning, stronger augmentation, localization, richer metrics, full-dataset evaluation, and domain-specific language generation.

Keywords

Aircraft damage detection; dent classification; crack classification; VGG16; transfer learning; BLIP; image captioning; computer vision; aviation maintenance; multimodal AI

Contents

- 1. Introduction
- 2. Related Technical Background
- 3. System Overview
- 4. Dataset and Preprocessing
- 5. Classification Model
- 6. BLIP Captioning and Summary Generation
- 7. End-to-End Inference Workflow
- 8. Experimental Setup and Results
- 9. Code-Level Explanation: Why Each Block Exists
- 10. Critical Analysis and Limitations
- 11. Recommended Improvements
- 12. Conclusion
- References
- Appendix A - Representative Code Walkthrough

1. Introduction

Aircraft visual inspection is a high-value computer vision problem because maintenance teams must identify surface defects reliably while dealing with a large number of images and variable lighting, viewpoints, materials, and defect shapes. The repository studied here frames the problem as an image classification task for two surface-damage classes - dent and crack - and then adds a vision-language component that converts the image into natural-language text. The

project description explicitly presents the system as a combination of computer vision for classification and a vision-language model for descriptive reporting. [1]

The central design decision is to separate recognition from language generation. VGG16 is responsible for a structured decision: which of the two damage categories is present? BLIP is responsible for a different output space: how can the image be described in words? This separation is useful because classification and captioning have different objectives and different training requirements.

The repository is particularly notable as a transfer-learning exercise. Instead of learning low-level image features from a small aircraft-specific dataset from scratch, it reuses the visual representation learned by VGG16 on ImageNet and freezes the base feature extractor. Only the new classifier layers are trained for the dent/crack task. The notebook then wraps BLIP inside a custom Keras layer so that a TensorFlow-centric notebook can call a PyTorch-based Hugging Face generation model. The code and logged outputs used for this analysis come directly from the public notebook in the repository. [1,2,3]

2. Related Technical Background

2.1 VGG16 and Transfer Learning

VGG16 is a deep convolutional neural network introduced by Simonyan and Zisserman. Its key architectural idea is stacking small 3x3 convolution filters in a 16-weight-layer network, producing strong visual representations that generalize beyond the original ImageNet task. [4]

In the repository, VGG16 is loaded with ImageNet weights and `include_top=False`, which removes the original ImageNet classification head. The network therefore acts as a feature extractor rather than as a 1000-class classifier. The final feature map is flattened and sent to two newly initialized 512-unit dense layers, each followed by 30% dropout, and then a one-unit sigmoid output. This is an appropriate transfer-learning pattern for a small binary dataset because it reduces the number of trainable parameters and avoids discarding useful generic visual features.

2.2 BLIP for Vision-Language Generation

BLIP (Bootstrapping Language-Image Pre-training) is a vision-language framework designed to support both image understanding and generation. The original work describes a pretraining approach that bootstraps captions and filters noisy web supervision, enabling strong performance across image-text retrieval, image captioning, and visual question answering. [5]

The repository uses the pretrained model identifier `Salesforce/blip-image-captioning-base`. The code provides two textual modes by changing the prompt: "This is a picture of" for caption mode and "This is a detailed photo showing" for summary mode. Consequently, the implementation should be understood as prompt-conditioned generation, not as a separately trained technical summarization model. The generated text is not grounded by a domain-specific aviation report schema, so its language output should be treated as descriptive assistance rather than a certified maintenance conclusion.

3. System Overview

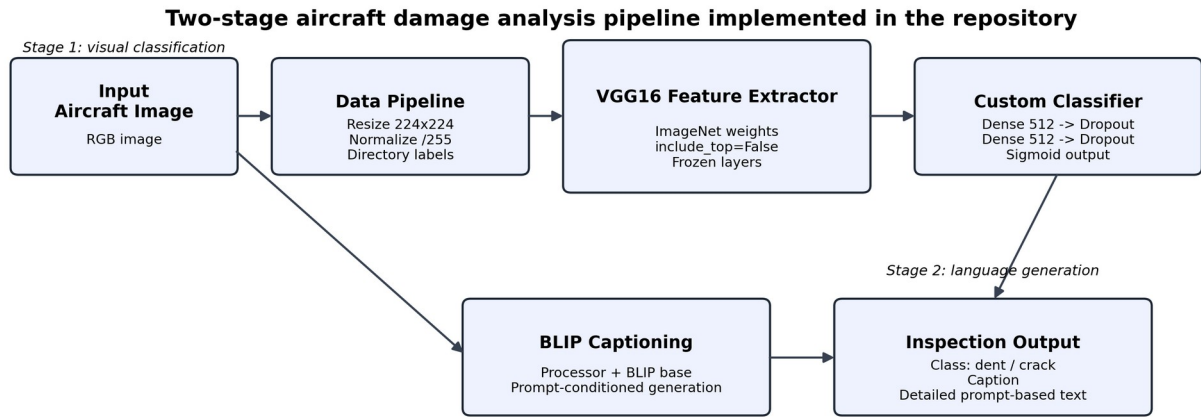


Figure 1. Reconstructed architecture of the two-stage pipeline from the repository notebook.

The system can be interpreted as the following sequence:

- An aircraft image is supplied as the input.
- The image is resized to 224x224 pixels and normalized by dividing pixel values by 255.
- The VGG16 convolutional base extracts visual features. Its layers are frozen, so the feature extractor is not updated during the five training epochs.
- The custom dense classifier converts those features into a probability for the binary dent/crack decision.
- The same or another aircraft image can be passed to BLIP through a custom Keras layer for text generation.
- The final output is therefore multimodal: a damage class plus one or more natural-language descriptions.

4. Dataset and Preprocessing

4.1 Data Source and Organization

The notebook downloads an aircraft damage tar archive from the Skills Network cloud-storage URL and states that its original source is a Roboflow aircraft-damage dataset. The Roboflow project lists two classes, crack and dent, and is distributed under CC BY 4.0. [1,6]

The notebook organizes the extracted data into the following directory layout:

```

aircraft_damage_dataset_v1/
├── train/
│   ├── dent/
│   └── crack/
├── valid/
│   ├── dent/
│   └── crack/

```

```
└─ test/
  └─ dent/
    └─ crack/
```

The recorded loader outputs show 300 training images, 96 validation images, and 50 test images. These counts are the experimental snapshot in the notebook and should be reported as such rather than assumed to describe the full upstream Roboflow collection, which is much larger. [1,6]

4.2 Preprocessing

```
train_datagen = ImageDataGenerator(rescale=1./255)
valid_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)
```

The code creates three `ImageDataGenerator` instances. Their only transformation is rescaling. The training generator additionally shuffles the training examples, while validation and test generators disable shuffling so that sample ordering remains deterministic for evaluation and visualization. The notebook description mentions augmentation as a general capability of `ImageDataGenerator`, but this specific implementation does not use rotations, flips, zooms, or other augmentation operations.

4.3 Generator Semantics

```
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(img_rows, img_cols),
    batch_size=batch_size,
    seed=seed_value,
    class_mode='binary',
    shuffle=True
)
```

`flow_from_directory` is used because the directory names themselves define the labels. Setting `class_mode="binary"` is sufficient because there are exactly two classes. `target_size=(224, 224)` makes each image compatible with the VGG16 input dimensions selected by the notebook, while `batch_size=32` controls memory use and gradient-update frequency.

5. Classification Model

5.1 Loading VGG16

```
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(img_rows, img_cols, 3)
)
```

`weights="imagenet"` imports a model that has already learned general visual features. `include_top=False` removes the original ImageNet classifier. `input_shape=(224,224,3)` establishes the expected RGB image dimensions. The choice allows the notebook to leverage transfer learning

instead of learning edge, texture, and shape features from only a few hundred aircraft examples. [4]

5.2 Freezing and Feature Extraction

```
output = base_model.layers[-1].output
output = keras.layers.Flatten()(output)
base_model = Model(base_model.input, output)
```

```
for layer in base_model.layers:
    layer.trainable = False
```

The code converts the final VGG16 convolutional output into a flat vector so the dense classifier can process it. The explicit loop setting trainable=False is important: it prevents the pretrained convolutional weights from changing during backpropagation. With the small dataset used here, this choice lowers the risk of catastrophic overfitting and substantially reduces training cost. A trade-off is that the learned representation cannot adapt to aircraft-specific textures and defect morphology.

5.3 Custom Classification Head

```
model = Sequential()
model.add(base_model)
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))
```

The first Dense layer maps the frozen feature vector into a 512-dimensional task-specific representation. ReLU provides nonlinear transformation. Dropout(0.3) randomly disables 30% of activations during training, which is intended to reduce co-adaptation and overfitting. A second 512-unit Dense layer provides additional nonlinear capacity. The final sigmoid returns a scalar probability p in $[0,1]$, which is naturally suited to binary classification. A threshold of 0.5 is later used to convert that probability into class 0 or class 1.

5.4 Optimization Objective

```
model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Adam is selected as the optimizer because it combines adaptive learning rates with momentum-like first- and second-moment estimates. The learning rate $1e-4$ is conservative, which is reasonable when training a small classification head over a frozen pretrained backbone. Binary cross-entropy is the standard loss for a sigmoid-based binary classifier. Accuracy is tracked as an easy-to-interpret metric, although it is not sufficient by itself for a safety-oriented defect detection study.

6. BLIP Captioning and Summary Generation

6.1 Loading the Model

```
processor = BlipProcessor.from_pretrained(
    "Salesforce/blip-image-captioning-base"
)
blip_model = BlipForConditionalGeneration.from_pretrained(
    "Salesforce/blip-image-captioning-base"
)
```

BlipProcessor handles image/token preprocessing and decoding, while BlipForConditionalGeneration performs autoregressive text generation conditioned on the image and text prompt. The repository relies on pretrained weights rather than fine-tuning BLIP for aviation language. [5]

6.2 Custom Keras Layer

```
class BlipCaptionSummaryLayer(tf.keras.layers.Layer):
    def __init__(self, processor, model, **kwargs):
        super().__init__(**kwargs)
        self.processor = processor
        self.model = model

    def call(self, image_path, task):
        return tf.py_function(
            self.process_image, [image_path, task], tf.string
        )
```

Subclassing `tf.keras.layers.Layer` creates a reusable bridge between TensorFlow/Keras and the external BLIP generation code. The `call` method uses `tf.py_function` because BLIP generation is implemented through the Python/PyTorch ecosystem rather than as a native TensorFlow graph operation. This is pragmatic for an educational notebook, but it also means the text-generation portion is not a normal differentiable Keras graph and is not intended to participate in end-to-end gradient-based training.

6.3 Image Processing and Prompt Logic

```
image_path_str = image_path.numpy().decode("utf-8")
image = Image.open(image_path_str).convert("RGB")
if task.numpy().decode("utf-8") == "caption":
    prompt = "This is a picture of"
else:
    prompt = "This is a detailed photo showing"
inputs = self.processor(images=image, text=prompt, return_tensors="pt")
output = self.model.generate(**inputs)
result = self.processor.decode(output[0], skip_special_tokens=True)
```

The image path arrives as a TensorFlow tensor, so `.numpy().decode("utf-8")` converts it to a Python string. PIL opens the image and converts it explicitly to RGB to avoid mode mismatches. The task string selects one of two prompts, and the prompt is fed together with the image to BLIP. `model.generate()` produces tokens, and `processor.decode()` converts the generated token IDs into human-readable text.

A crucial interpretive point is that the code does not train a domain-specific "summary model." Summary mode is created by changing the prompt, while the same pretrained BLIP model performs generation in both modes. Therefore, the quality of the so-called summary depends heavily on what BLIP learned from its general image-text training and on the prompt wording.

7. End-to-End Inference Workflow

```
def generate_text(image_path, task):
    blip_layer = BlipCaptionSummaryLayer(
        processor=processor, model=blip_model
    )
    result = blip_layer(image_path, task)
    return result
```

The helper function is intentionally simple: it instantiates the wrapper layer, passes the image path and task, and returns the text tensor. In practical deployment, this function could be called after the classifier predicts dent/crack, allowing the application to show a class label followed by an AI-generated description.

The repository example then calls `generate_text` twice: once with `task="caption"` and once with `task="summary"`. The notebook records an example caption equivalent to a general plane description and a summary equivalent to a detailed aircraft-fuselage damage description. These examples demonstrate the intended user experience but should not be interpreted as quantitative evidence of technical-report accuracy. [1]

8. Experimental Setup and Results

8.1 Experimental Configuration

Parameter	Setting
Training images	300
Validation images	96
Test images	50
Input size	224 x 224 x 3
Batch size	32
Epochs	5
Backbone	VGG16, ImageNet pretrained, frozen
Classifier	512 -> Dropout(0.3) -> 512 -> Dropout(0.3) -> 1 sigmoid
Optimizer	Adam, learning rate 0.0001
Loss	Binary cross-entropy
Threshold for prediction	0.5
Text model	Salesforce/blip-image-captioning-base

Table 1. Experimental configuration reconstructed from the notebook.

8.2 Training Curves

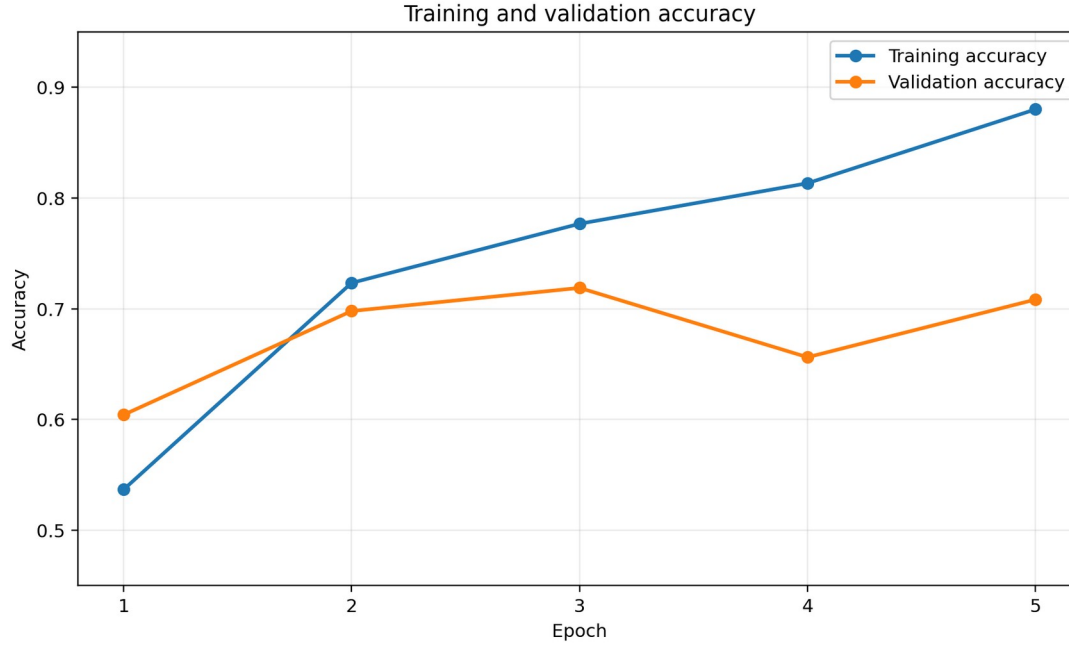


Figure 2. Accuracy values recorded during the five training epochs.

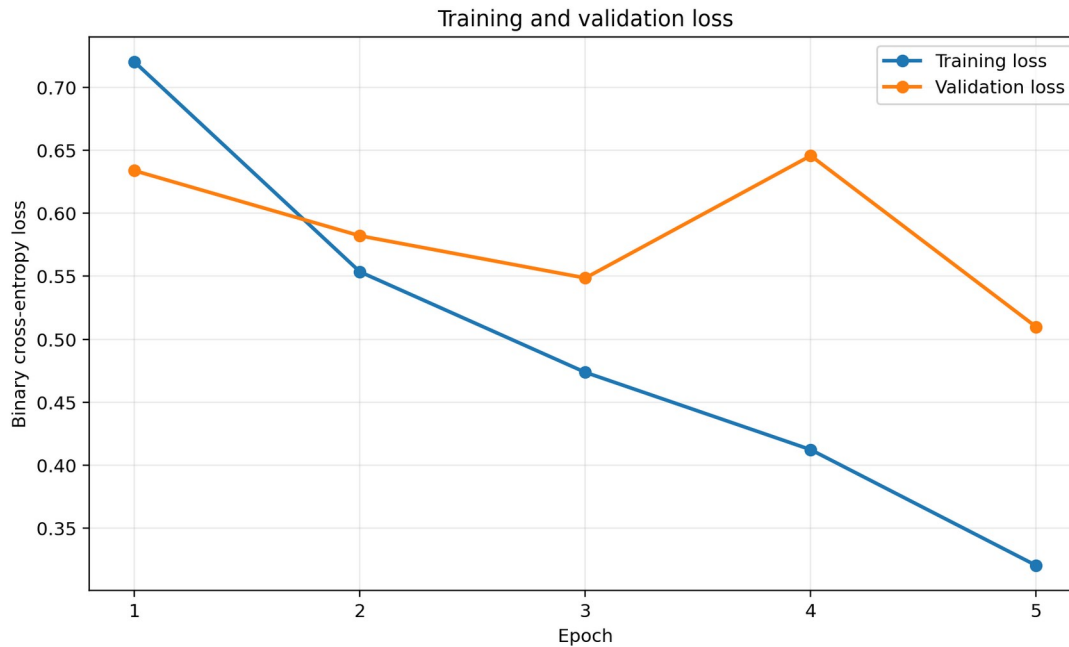


Figure 3. Binary cross-entropy loss values recorded during the five training epochs.

The learning curves show a clear divergence between training and validation performance. Training accuracy reaches 88.00% by epoch 5, while validation accuracy fluctuates and ends at 70.83%. The validation peak of 71.88% occurs at epoch 3 before dropping at epoch 4 and partially recovering at epoch 5. This pattern is consistent with some degree of overfitting: the model continues to fit the training data while the validation set does not improve monotonically.

8.3 Reported Test Result

Dataset / evaluation snapshot from the notebook		Reported result
Training images	300	
Validation images	96	
Test images	50	
Epochs	5	
Batch size	32	
Test loss	0.7326	
Test accuracy	0.6875 (68.75%)	

Figure 4. Dataset counts and evaluation snapshot from the executed notebook.

The notebook reports test loss = 0.7326 and test accuracy = 0.6875 (68.75%). However, the evaluation code is:

```
test_loss, test_accuracy = model.evaluate(
    test_generator,
    steps=test_generator.samples // test_generator.batch_size
)
```

With 50 test samples and a batch size of 32, integer division produces 1 step. Therefore, the displayed accuracy corresponds to one complete 32-image batch rather than all 50 test images. The notebook result is still a valid execution result for that call, but it should not be reported as a full-test-set accuracy without this qualification. A more appropriate evaluation would omit the steps argument or use `ceil(50/32)` while ensuring the generator handles the final partial batch.

9. Code-Level Explanation: Why Each Block Exists

Code block	Why it is there / what it accomplishes
Warnings suppression	Hides non-fatal warning noise in the notebook. It improves readability but can make dependency or deprecation issues less visible during development.
Random seeds	Sets Python, NumPy, and TensorFlow seeds to 42. This improves reproducibility of data order and stochastic training operations, although exact reproducibility can still vary across hardware/software stacks.
Configuration	batch_size=32 and n_epochs=5 provide a bounded educational experiment. 224x224 matches the selected VGG16 input tensor.
Dataset download/extraction	Automates setup so the notebook can create a clean local dataset without manual file management.
Directory paths	Separates train, validation, and test locations and

	keeps subsequent generator code readable.
ImageDataGenerator	Rescales image pixels into a compact numerical range. The train generator shuffles; validation/test do not.
flow_from_directory	Maps folder names to binary labels and batches data lazily, avoiding the need to load every image into RAM at once.
VGG16 load	Reuses ImageNet visual features instead of training all convolutional layers from scratch.
Flatten	Converts the final convolutional feature map into a vector that Dense layers can consume.
Freeze base model	Prevents pretrained weights from being modified, reducing trainable capacity and training cost.
Dense + ReLU	Adds task-specific nonlinear decision capacity for dent/crack classification.
Dropout	Regularizes the classifier and is intended to reduce overfitting on a small dataset.
Sigmoid output	Produces a probability for one of two classes.
Adam + binary cross-entropy	Matches the binary sigmoid objective and provides adaptive gradient updates.
model.fit	Trains the custom head using train batches while measuring validation performance after each epoch.
History extraction	Stores accuracy/loss sequences so learning curves can be plotted and interpreted.
Test evaluation	Measures held-out performance, although the current steps expression evaluates only one full batch.
Prediction visualization	Uses a 0.5 threshold, inverts class_indices back to class names, and displays true vs. predicted labels for qualitative inspection.
BLIP processor/model loading	Downloads pretrained multimodal weights and the corresponding preprocessing/tokenization logic.
Custom Keras layer	Provides a callable TensorFlow-facing wrapper for a PyTorch/Transformers text generator.
tf.py_function	Allows ordinary Python code to execute inside the Keras layer boundary; convenient but not suited to graph compilation or differentiable training.
Prompt switch	Creates caption and summary modes without training separate models.
generate_text helper	Provides a compact user-facing function so downstream code does not need to instantiate BLIP components repeatedly.

Table 2. Code rationale for the principal executable blocks in the notebook.

10. Critical Analysis and Limitations

10.1 What the Project Does Well

- Uses transfer learning effectively for a small labeled dataset rather than attempting to learn a deep convolutional representation from scratch.

- Separates classification from language generation, making the architecture easy to reason about and extend.
- Uses deterministic validation/test generators and a fixed seed, which supports clearer debugging and repeatable experiments.
- Provides qualitative prediction visualization, not just a scalar accuracy number.
- Demonstrates a practical multimodal bridge between TensorFlow/Keras and the PyTorch/Hugging Face ecosystem.

10.2 Limitations That Matter Technically

- The experimental snapshot is small: 300 training images, 96 validation images, and 50 test images.
- Despite discussion of augmentation, the actual ImageDataGenerator configuration only rescales; it does not perform geometric or photometric augmentation.
- The VGG16 convolutional backbone is completely frozen, which may limit adaptation to aircraft-specific defect textures.
- Only five epochs are used, making the experiment more suitable as a prototype than a final model-selection study.
- Accuracy is the only quantitative metric shown. Precision, recall, F1-score, confusion matrix, ROC-AUC, and class-specific error analysis are absent.
- The test evaluation uses `test_samples // batch_size`, so 18 of the 50 test images are omitted from the reported metric in that call.
- The classifier does not localize damage. It classifies the entire image, so it cannot answer where the defect is or how large it is.
- The BLIP "summary" is prompt-conditioned caption generation, not a domain-specific summarization or maintenance-report model.
- The BLIP output is not explicitly conditioned on the classifier prediction. It may describe an aircraft image without correctly naming the model-predicted damage class.
- The notebook includes dependency-resolution warnings during setup, indicating that a clean reproducible environment should pin compatible package versions rather than relying on a mutable notebook environment.

10.3 Reproducibility Observation

The notebook shows a sequence of package installations and resolver warnings involving NumPy, TensorFlow, ml-dtypes, OpenCV, transformers, and other packages. This does not mean the method itself is invalid, but it does mean that reproducing the exact environment may require explicit version pinning. A production version should use a requirements file or lock file with tested combinations of Python, TensorFlow/Keras, Transformers, PyTorch, Pillow, and NumPy.

11. Recommended Improvements

Improvement	Technical change	Expected benefit
Full test evaluation	Evaluate the complete test generator, not <code>floor(test_samples/batch_size)</code> .	Unbiased test metric over all test images.
Data augmentation	Add flips where physically valid,	Improves robustness to viewpoint

	small rotations, crops/zoom, brightness/contrast changes, and controlled blur.	and lighting variation.
Fine-tune VGG16	Unfreeze only the last few convolution blocks after classifier warm-up.	Allows adaptation to aircraft-specific visual cues while retaining general features.
Better metrics	Report precision, recall, F1, confusion matrix, balanced accuracy, and ROC-AUC where appropriate.	Reveals class-specific failure modes hidden by accuracy.
Localization	Replace or augment the classifier with an object detector or segmentation model.	Identifies defect location and extent rather than only whole-image class.
Domain-specific language generation	Condition BLIP or a stronger VLM on classifier output and structured fields such as defect type, region, and confidence.	Produces more consistent inspection reports.
Confidence calibration	Calibrate probabilities and report uncertainty.	Supports human review and reduces overconfident automation.
Versioned environment	Pin compatible Python/library versions in requirements.txt or a lock file.	Improves reproducibility and deployment reliability.
Human-in-the-loop workflow	Require inspector confirmation for uncertain or high-risk predictions.	Makes the system safer for real maintenance use.

Table 3. Recommended next steps for moving from an educational prototype toward a stronger research system.

12. Conclusion

The Aircraft-Dent-Detector repository demonstrates a clear and accessible multimodal AI pattern: transfer learning for visual damage classification followed by pretrained vision-language generation. The VGG16 stage is computationally economical and conceptually appropriate for a small binary dataset, while BLIP adds an intuitive language interface that can make predictions easier for a human to interpret. The repository therefore succeeds as a learning-oriented prototype that connects CNN-based feature extraction, binary classification, evaluation, visualization, and vision-language generation in one workflow. [1,4,5]

At the same time, the analysis shows why the current system should not be presented as a complete autonomous aircraft-inspection solution. Its reported 68.75% test accuracy comes from a one-batch evaluation, its data augmentation is minimal, its backbone is frozen, and its text generator is general-purpose rather than aviation-specific. The most important path forward is to turn the demonstration into a better controlled experiment: evaluate every test image, expand and stratify the dataset, introduce physically meaningful augmentation, fine-tune the visual backbone, report class-level metrics, and add localization. The language stage should then be grounded in the classifier result and structured inspection metadata.

References

- [1] Barman, M. Aircraft-Dent-Detector. GitHub repository and notebook analyzed for this paper.
<https://github.com/emManab/Aircraft-Dent-Detector> (accessed 22 Aug 2026).
- [2] TensorFlow/Keras documentation and implementation conventions for VGG16 transfer learning and ImageDataGenerator.
- [3] Hugging Face Transformers implementation of BLIP, including BlipProcessor and BlipForConditionalGeneration.
- [4] Simonyan, K., & Zisserman, A. (2014/2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556 / ICLR 2015. <https://arxiv.org/abs/1409.1556>
- [5] Li, J., Li, D., Xiong, C., & Hoi, S. (2022). BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. Proceedings of ICML 2022, PMLR 162, 12888-12900.
<https://proceedings.mlr.press/v162/li22n.html>
- [6] Youssef Donia. Aircraft Damage Detection Dataset. Roboflow Universe, CC BY 4.0.
<https://universe.roboflow.com/youssef-donia-fhktl/aircraft-damage-detection-1j9qk>

Appendix A - Representative Code Walkthrough

The following compact walkthrough follows the executable order of the notebook. It focuses on the purpose of the code and the design reasoning behind each choice rather than reproducing the entire notebook verbatim.

A1. Reproducibility and configuration

```
seed_value = 42
random.seed(seed_value)
np.random.seed(seed_value)
tf.random.set_seed(seed_value)

batch_size = 32
n_epochs = 5
img_rows, img_cols = 224, 224
```

Why: Sets reproducible random states and centralizes core hyperparameters.

A2. Dataset acquisition

```
urllib.request.urlretrieve(url, tar_filename)
with tarfile.open(tar_filename, "r") as tar_ref:
    tar_ref.extractall()
```

Why: Automates downloading and extraction. In a hardened production implementation, archive members should be validated before extraction.

A3. Input pipeline

```
ImageDataGenerator(rescale=1./255)
flow_from_directory(..., class_mode="binary")
```

Why: Normalizes raw pixel ranges and maps directory names to binary labels.

A4. Frozen VGG16

```
VGG16(weights="imagenet", include_top=False, input_shape=(224,224,3))
for layer in base_model.layers:
    layer.trainable = False
```

Why: Reuses generic visual features while training only the task-specific head.

A5. Classifier head

```
Dense(512, activation="relu")
Dropout(0.3)
Dense(512, activation="relu")
Dropout(0.3)
Dense(1, activation="sigmoid")
```

Why: Creates a binary decision function with nonlinear capacity and regularization.

A6. Training

```
history = model.fit(
    train_generator,
    epochs=5,
    validation_data=valid_generator
)
```

Why: Updates the trainable classifier head and stores the learning history.

A7. Prediction threshold

```
predictions = model.predict(test_images)
predicted_classes = (predictions > 0.5).astype(int)
```

Why: Turns continuous sigmoid probabilities into binary class IDs for display.

A8. BLIP

```
processor = BlipProcessor.from_pretrained(...)
blip_model = BlipForConditionalGeneration.from_pretrained(...)
```

Why: Loads a general-purpose image-to-text model and its processor.

A9. Keras/PyTorch bridge

```
return tf.py_function(self.process_image, [image_path, task], tf.string)
```

Why: Allows the TensorFlow-facing layer to call ordinary Python/PyTorch generation logic.

A10. Prompt-controlled output

```
if task == "caption":
    prompt = "This is a picture of"
else:
    prompt = "This is a detailed photo showing"
```

Why: Creates two user-facing output modes without retraining the language model.

Appendix B - Research Interpretation in One Page

- Task: binary image classification of aircraft surface damage into dent and crack.

- Backbone: ImageNet-pretrained VGG16, used as a frozen feature extractor.
- Head: two 512-unit ReLU layers with 30% dropout and one sigmoid output.
- Optimization: Adam at $1e-4$ with binary cross-entropy.
- Data snapshot: 300 train / 96 validation / 50 test images.
- Training snapshot: 5 epochs, with training accuracy rising to 88.00% and validation accuracy ending at 70.83%.
- Evaluation caveat: the displayed 68.75% test accuracy is based on one 32-image batch because of integer division in the evaluation call.
- Language stage: Salesforce BLIP image-captioning-base, wrapped in a custom Keras layer using `tf.py_function`.
- Important semantic caveat: the "summary" mode is prompt-conditioned BLIP generation, not an independently trained aviation summarizer.
- Research maturity: a strong educational prototype, not a production-certified autonomous aircraft inspection system.